

Track A Chapter — Candidate Proposal Generation and Head Manifest Construction

This chapter documents Track A of the Ego4D-LiteSTA pipeline as both a theoretical exposition and a practical lab manual. It is intended to be pasted into a thesis with minimal editing while remaining executable for local experiments. The emphasis is on explaining why each design choice exists (sampling strategy, proposal pruning, recall measurement) and how to reproduce every artifact required by downstream Tracks B and C.

Table of Contents

1. Purpose, Scope, and Positioning
 2. Problem Setting and Notation
 3. Data Assets and Directory Layout
 4. Conceptual View of Track A
 5. Stage A Theory — Proposal Generation
 6. Stage A Practice — Configuration, Execution, Outputs
 7. Stage A Validation and Failure Modes
 8. Stage B Theory — Cropping, Label Fusion, and Recall
 9. Stage B Practice — Configuration, Execution, Outputs
 10. Stage B Validation and Failure Modes
 11. Head Manifests and Semantic Fields
 12. End-to-End Workflows (from smoke test to production runs)
 13. Experiment Design and Ablations (K-sweeps, oracle vs YOLO)
 14. Reproducibility, Logging, and Run Naming
 15. Performance, Resource Management, and Determinism
 16. Troubleshooting Playbook
 17. Appendix A — Parameter Reference (Stage A)
 18. Appendix B — Parameter Reference (Stage B)
 19. Appendix C — File Schemas and Examples
 20. Appendix D — Environment Variable Overrides (Stage A)
 21. Appendix E — Checklists and Sanity Tests
 22. Appendix F — Integration Notes for Track B and Track C
-

1. Purpose, Scope, and Positioning

- Track A is the entry point for the LiteSTA pipeline. It converts raw ego-centric frames into structured proposals and head manifests that can be consumed by Track B (temporal head training) and Track C (pruning/selection).
- The design is intentionally local-first: all inputs (frames, labels, manifests) and outputs (runs) live under `local_extraction/`.
- The chapter balances two concerns:
 - **Theoretical justification:** why sampling only the last frame can work, how IoU-based recall relates to downstream training, and how proposal priors influence TTC (time-to-contact)

regression heads.

- **Practical reproducibility:** exact paths, commands, configuration defaults, and file schemas.
- Scope includes Stage A (detector/proposal generation) and Stage B (crop extraction, recall computation, manifest writing). Training of the temporal head itself belongs to Track B, but this chapter explains how Track A artifacts flow into it.

2. Problem Setting and Notation

- Goal: detect hand-centric regions in ego-centric video frames and prepare balanced candidate sets with optional semantic labels (verb, noun, TTC) to supervise temporal reasoning heads.
- Input: a set of frames per unique video identifier `uid`, stored as JPEGs with zero-padded frame indices.
- Output: proposals (Stage A), cropped regions and manifests (Stage B), and optional recall metrics (Stage B evaluation).
- Notation (used throughout the chapter):
 - `Iu,f` : frame image for uid `u` at frame index `f`.
 - `Bu,f` : list of bounding boxes for `Iu,f`, each box `b = (x1, y1, x2, y2, conf, cls)`.
 - `K` : maximum number of proposals kept per image.
 - `IoU(b, g)` : intersection-over-union between candidate `b` and ground-truth `g`.
 - `Recall@K` : fraction of images with ground-truth where at least one candidate achieves `IoU ≥ τ`.
 - `τ` : IoU threshold (default 0.5).
- Assumption: downstream heads learn from crops plus optional semantics; higher recall@K on Stage A proposals reduces missed positives for Track B.

3. Data Assets and Directory Layout

- Frames: `local_extraction/v2/extracted_frames/<uid>/<frame:07d>.jpg`
- YOLO-format labels for oracle/evaluation:
`local_extraction/v2/yolo_labels_540/{clips|videos}/<uid>/<frame:07d>.txt`
- Semantic head manifests (optional):
`local_extraction/v2/manifests/head*_<clip|video>.json`
- Run outputs: `local_extraction/runs/Track_A/`
- Configuration: `local_extraction/configs/trackA.yaml` (+ inherits `local_extraction/configs/base.yaml`)
- Core scripts:
 - `local_extraction/trackA/trackA_stageA/trackA_stageA.py`
 - `local_extraction/trackA/trackA_stageB/trackA_stageB.py`
 - Utilities: `local_extraction/trackA/trackA_stageB/san_stageB.py`,
`local_extraction/trackA/trackA_stageA/oracle_k_sweep.py`,
`local_extraction/trackA/trackA_stageA/sweep_k_metrics.py`,
`local_extraction/trackA/trackA_plots.py`
- Virtual environment (Windows PowerShell):

```
cd D:\Thesis\Ego4d-LiteSTA
.\local_extraction\.venv\Scripts\Activate.ps1
```

4. Conceptual View of Track A

- Track A is a two-stage filter:
 - **Stage A:** Generate spatial proposals per frame using either YOLO inference or oracle labels. Choices here control recall and noise.
 - **Stage B:** Convert proposals into crops, join with semantics (if available), and compute recall statistics to quantify coverage.
- Design rationale:
 - Defer temporal reasoning to Track B; keep Track A purely spatial to simplify scaling and debugging.
 - Allow oracle mode to isolate proposal quality from detector performance and to measure the intrinsic ceiling given available labels.
 - Use JSONL + CSV outputs to support both scripted and ad-hoc analysis.
- The chapter treats Track A as a reproducible experiment: each run is identified by its prefix and timestamp, and summaries are written to JSON for later aggregation.

5. Stage A Theory — Proposal Generation

- Objective: approximate the unknown distribution of hand-centric boxes with a manageable candidate set of size `K` per image.
- Two modes reflect two priors:
 - **YOLO mode:** learns a prior over hand locations from data; introduces detector bias (precision/recall trade-off) and is sensitive to confidence and NMS thresholds.
 - **Oracle mode:** reuses ground-truth YOLO-format labels as proposals; represents an upper bound on recall for a given `K`.
- Sampling strategy:
 - Default `last_frame_only = True` ensures exactly one image per `uid`, aligning with Track B's assumption of per-clip temporal localization anchored at the final frame.
 - Setting `last_frame_only = False` expands coverage across frames but increases runtime and storage; useful for ablations on temporal diversity.
- Bounding box conversion:
 - Oracle mode reads normalized YOLO labels `(cls, cx, cy, w, h)` and denormalizes using image width/height to pixel coordinates `(x1, y1, x2, y2)`.
 - YOLO mode loads Ultralytics weights once and predicts boxes per image; outputs are already in pixel coordinates.
- Ranking and truncation:
 - Boxes are sorted by confidence and truncated to `K` to bound downstream computation; this is the main controllable lever for recall vs efficiency.
- Evaluation intent:
 - Stage A itself does not compute recall; it produces the candidates consumed by Stage B where recall is measured against the same label set.
- Determinism and seed:
 - The Ultralytics pipeline can be made deterministic via global seeds, but Stage A primarily depends on deterministic file ordering and fixed thresholds; no randomness is introduced in oracle mode.

6. Stage A Practice — Configuration, Execution, Outputs

- Configuration source: `local_extraction/configs/trackA.yaml`
 - `stage_a.mode` : "yolo" or "oracle"
 - `stage_a.k` : integer K
 - `stage_a.last_frame_only` : true/false
 - `stage_a.max_images` : null or integer cap
 - `stage_a.yolo.weights` : path to Ultralytics checkpoint (default points to `toolkit_yolo/runs/.../weights/best.pt`)
 - `stage_a.yolo.imgsz` : integer resize for inference (default 960)
 - `stage_a.yolo.conf_thresh` : detection confidence threshold (default 0.05)
 - `stage_a.yolo.nms_iou` : NMS IoU threshold (default 0.45)
 - `stage_a.oracle.label_space` : "clips" or "videos" (matches subfolder under `yolo_labels_540`)
 - `stage_a.output.save_per_image_csv` : false/true
 - `stage_a.output.run_prefix` : default "trackA_stageA" ; controls run folder name
- Execution (from repo root):

```
python local_extraction/trackA/trackA_stageA/trackA_stageA.py
```

- Environment overrides (optional, see Appendix D):
 - `STAGEA_MODE` , `STAGEA_K` , `STAGEA_MAX_IMAGES` , `STAGEA_DEMO_MODE` , `STAGEA_DEMO_N`
 - Use overrides for quick sweeps without editing YAML.
- Input expectations:
 - Frames exist under `local_extraction/v2/extracted_frames/` .
 - Oracle mode additionally expects matching label txt files under `local_extraction/v2/yolo_labels_540/{clips|videos}/` .
- Output layout (per run):

```
local_extraction/runs/Track_A/trackA_stageA_<timestamp>/
├─ candidates.jsonl    # JSON lines with {uid, frame, boxes:
                      [{x1,y1,x2,y2,conf,cls}]}
├─ candidates/        # Per-image CSVs if save_per_image_csv=true
└─ summary.json       # Runtime metadata and detector configuration
```

- Sample `candidates.jsonl` record:

```
{"uid":"00be...1705","frame":99,"boxes":
[{"x1":161.8,"y1":0.3,"x2":373.9,"y2":244.8,"conf":1.0,"cls":26}]}
```

- Summary contents:
 - Image count and average boxes per image.
 - Mode (`yolo` or `oracle`), `K` , label space.
 - Paths for frames and labels.
 - Duration and run stamp.
 - YOLO configuration snapshot when applicable.
- Typical runtime guidance:
 - Oracle mode is I/O bound; expect throughput proportional to disk speed.
 - YOLO mode loads the model once; batch size is effectively 1 per image in the current implementation; adjust `K` and `max_images` for quick tests.

7. Stage A Validation and Failure Modes

- Validation steps:
 - Confirm `candidates.jsonl` exists and has non-empty `boxes` entries.
 - Spot-check a few frames: open an image and overlay `boxes` to ensure coordinate correctness (e.g., negative values should be clamped later in Stage B).
 - Inspect `summary.json` to verify that the intended mode and `K` were applied.
- Common failure modes and mitigations:
 - **No images found:** the configured frames root is empty or mispointed; verify `version` and path in `trackA.yaml` .
 - **YOLO import error:** install `ultralytics` in the active venv or switch to oracle mode temporarily.
 - **Zero boxes in oracle mode:** label txt files are missing or malformed; ensure `label_space` matches `clips` vs `videos` .
 - **Excessive boxes:** reduce `K` or raise `conf_thresh` to control downstream storage.
 - **Performance bottleneck:** enable `max_images` or `DEMO_MODE` for smoke tests; lower `imgsz` if acceptable.
- Quality heuristics:
 - High `avg_boxes_per_image` with low recall in Stage B often indicates misaligned labels (e.g., wrong space) or overly generous thresholds.
 - If the same frames are reused across runs, keep the run prefix descriptive to avoid confusion in auto-detection by Stage B.

8. Stage B Theory — Cropping, Label Fusion, and Recall

- Objective: transform proposals into training-ready crops and measure how well the proposals cover ground-truth boxes.
- Inputs:
 - `candidates.jsonl` from Stage A (auto-detected latest run if unspecified).
 - Frame images (same root as Stage A).
 - Optional YOLO-format labels for recall computation.
 - Optional semantic head manifests (`head_train_*` , `head_val_*`) that provide verb, noun, and TTC annotations.
- Operations performed:
 - For each candidate box, clamp coordinates to image bounds to prevent invalid crops.
 - Crop the region; optionally resize to a fixed `CROP_SIZE` (default `(256, 256)`).

- Emit per-crop metadata (JSONL + CSV) including hit flags if evaluation is enabled.
- Aggregate per-frame statistics for recall@K and mean best IoU.
- If semantic manifests exist, join candidates with ground-truth boxes to label candidates as positive/negative and propagate verb/noun/TTC.
- Why evaluation happens here:
 - Recall@K needs ground-truth; computing it after cropping ensures that geometric clamping and candidate truncation are reflected in the metric.
 - By writing recall metrics back into the Stage A summary, we preserve provenance and make later plots easier.
- Relation to Track B:
 - `head_train.jsonl` and `head_val.jsonl` produced here are the direct inputs to Track B training.
 - The manifest rows already encode positive/negative labels and TTC regression targets when semantics are available, simplifying the Track B dataloader.

9. Stage B Practice — Configuration, Execution, Outputs

- Configuration source: `local_extraction/configs/trackA.yaml`
 - `stage_b.candidates_source` : explicit path to `candidates.jsonl` or `null` to auto-detect the latest Stage A run.
 - `stage_b.crop_size` : `[W, H]` or `null` to keep native crop size.
 - `stage_b.keep_top_n` : limit number of candidates per image (after Stage A's `K`).
 - `stage_b.eval_with_labels` : `true/false` to enable recall computation using YOLO labels.
 - `stage_b.iou_thresh` : IoU threshold τ for hit determination (default 0.5).
 - `stage_b.manifests.use_clip_manifest` : choose clip-level vs video-level manifests (`true` selects clip).
 - `stage_b.manifests.train` / `stage_b.manifests.val` : explicit manifest paths.
 - `stage_b.output.write_head_train_val` : write consolidated `head_train.jsonl` and `head_val.jsonl` .
 - `stage_b.output.write_semantics_in_manifest` : include semantic fields in per-crop manifest rows when available.
 - `stage_b.output.write_back_to_stagea_summary` : mirror recall metrics into Stage A summary.
 - `stage_b.output.run_prefix` : default `"trackA_stageB"` .
 - Runtime caps: `runtime.max_images` , `demo.enabled` , `demo.max_samples` , `runtime.print_progress` .
- Execution (from repo root):

```
python local_extraction/trackA/trackA_stageB/trackA_stageB.py
```

- Input resolution:
 - If `candidates_source` is `null` , Stage B scans `local_extraction/runs/Track_A/` for the newest `candidates.jsonl` .

- Frames are loaded from the same `extracted_frames` root; missing images are skipped with a warning.
- Output layout (per run):

```
local_extraction/runs/Track_A/trackA_stageB_<timestamp>/
├─ crops/                    # Cropped ROI images
├─ manifest.csv              # Per-crop metadata (CSV)
├─ manifest.jsonl            # Per-crop metadata (JSONL)
├─ head_train.jsonl          # Consolidated positives/negatives for
training (if enabled)
├─ head_val.jsonl            # Consolidated positives/negatives for
validation (if enabled)
└─ summary.json              # Recall metrics and configuration
snapshot
```

- Manifest row fields:
 - `uid` , `frame` , `roi_idx`
 - `crop_path` (relative to run dir), `x1,y1,x2,y2` (clamped pixel coords)
 - `conf` , `cls` , `hit` (boolean; IoU $\geq \tau$ against YOLO labels when evaluation enabled)
 - Optional: `verb_id` , `noun_id` , `ttc` , `is_positive` , `iou` (only when semantics available and written)
- Summary contents:
 - Counts: images processed, crops produced.
 - Paths: frames root, labels root, candidates path.
 - Detector metadata mirrored from Stage A summary when present.
 - Recall metrics if computed: `images_with_gt` , `images_with_hit` , `recall_at_K` , `mean_best_iou` , histogram bins.
 - Head manifest bookkeeping: number of entries loaded and rows written.
- Duration considerations:
 - Cropping and resizing dominate runtime; performance scales with number of proposals (`K` * number of images).
 - Enabling evaluation adds negligible overhead relative to I/O.

10. Stage B Validation and Failure Modes

- Validation steps:
 - Ensure `manifest.jsonl` and `manifest.csv` are present and non-empty.
 - Check a few crops visually to confirm that clamping avoided negative coordinates and that resizing preserved aspect expectations.
 - If semantics are expected, verify that `head_train.jsonl` and `head_val.jsonl` exist and contain `is_positive` fields.
 - Read `summary.json` and confirm recall metrics are reasonable (non-zero when labels exist).
- Common failure modes and mitigations:
 - **Stage B cannot find candidates:** set `stage_b.candidates_source` explicitly to the intended `candidates.jsonl` .
 - **Empty crops or manifest:** Stage A produced zero boxes; rerun Stage A or adjust `K` .

- **Zero recall:** labels may be in the wrong `label_space` or misaligned with frames; confirm `clips` vs `videos` and file naming.
- **Missing semantics:** manifests may be absent or pointed to the wrong variant; verify `use_clip_manifest`.
- **Large disk footprint:** disable per-image CSVs in Stage A and reduce `K` or `keep_top_n` in Stage B.
- Quality heuristics:
 - If recall is high but `is_positive` counts are low, manifest linkage may be mismatched; inspect `(uid, frame)` keys in the head manifest files.
 - When crops appear empty or black, verify that images exist and that clamp logic is functioning; corrupt images will be skipped silently.

11. Head Manifests and Semantic Fields

- Purpose: encode supervisory signals (verb, noun, TTC) and binary labels for each candidate.
- Source files (JSON lists):
 - `head_train_clip.json` / `head_val_clip.json` or `head_train_video.json` / `head_val_video.json`
 - Each item contains `image`, `gt_box`, `verb_id`, `noun_id`, `ttc`.
- Joining logic:
 - Stage B maps each manifest item by `(uid, frame)` derived from the image path.
 - For each candidate crop belonging to that `(uid, frame)`, IoU is computed against `gt_box`; `is_positive = IoU >= τ`.
 - Semantics are propagated to manifest rows and consolidated head files.
- Output semantics:
 - `verb_id`, `noun_id`: categorical action components.
 - `ttc`: continuous time-to-contact value (seconds).
 - `ttc_bin`: not written by Stage B, but can be derived downstream if needed.
 - `is_positive`: boolean indicating spatial match.
- Impact on Track B:
 - Track B expects these fields to supervise classification and TTC regression heads. Absence of semantics results in binary-only training.

12. End-to-End Workflows (from smoke test to production runs)

- **First-time setup:**
 1. Activate venv: `.\local_extraction\.venv\Scripts\Activate.ps1`
 2. Verify imports:

```
python -c "from local_extraction.trackA.trackA_stageA import trackA_stageA; print('Stage A OK')"
```

```
python -c "from local_extraction.trackA.trackA_stageB import trackA_stageB; print('Stage B OK')"
```

3. Confirm data presence (PowerShell):


```
ls local_extraction/v2/extracted_frames | Measure-Object  
ls local_extraction/v2/manifests
```

- **Smoke test (oracle, small subset):**

1. Set overrides:

```
$env:STAGEA_MODE = "oracle"  
$env:STAGEA_K = "5"  
$env:STAGEA_MAX_IMAGES = "50"  
python local_extraction/trackA/trackA_stageA/trackA_stageA.py
```

2. Run Stage B with evaluation:

```
python local_extraction/trackA/trackA_stageB/trackA_stageB.py
```

3. Inspect `summary.json` for recall and mean IoU.

- **YOLO-based production run:**

1. Edit `stage_a.mode: "yolo"` and weights in `configs/trackA.yaml` .
2. Run Stage A:

```
python local_extraction/trackA/trackA_stageA/trackA_stageA.py
```

3. Run Stage B (auto-detect latest candidates):

```
python local_extraction/trackA/trackA_stageB/trackA_stageB.py
```

4. Optional: `python local_extraction/trackA/trackA_stageB/san_stageB.py` to summarize positives/negatives.

- **K-sweep (oracle):**

```
$env:ORACLE_SWEEP_KS = "4,6,8,10,12"  
$env:ORACLE_SWEEP_MAX_IMAGES = "500"  
python local_extraction/trackA/trackA_stageA/oracle_k_sweep.py
```

- Use this to estimate marginal gains of increasing `K`.

- **Metrics plots:**

```
python local_extraction/trackA/trackA_plots.py --summary
"local_extraction/runs/Track_A/trackA_stageB_*/summary.json" --out
"local_extraction/runs/Track_A/plots"
```

- **Hand-off to Track B:**

- Point Track B to the desired Stage B run, or rely on auto-discovery of the latest run via `latest_stageB_run` logic inside `trackB_dataset.py`.
- Ensure `head_train.jsonl` and `head_val.jsonl` exist for supervised training; otherwise Track B will operate with binary cues only.

13. Experiment Design and Ablations (K-sweeps, oracle vs YOLO)

- `oracle_k_sweep.py` :
 - Runs Stage A (oracle) across multiple `K` values, then Stage B to compute recall for each.
 - Use environment variables `ORACLE_SWEEP_KS` and `ORACLE_SWEEP_MAX_IMAGES` to control the sweep.
 - Output: CSV/JSON under `local_extraction/runs/Track_A/KSweep/StageA_KSweep_<timestamp>/`.
- `sweep_k_metrics.py` :
 - Extends sweeps to YOLO inference; useful to see how detector confidence interacts with `K`.
 - Produces `metrics.csv` and `summary.json` with recall curves.
- Ablation suggestions:
 - **Last-frame vs full-clip:** toggle `last_frame_only` to understand how many frames contribute unique positives.
 - **IoU threshold sensitivity:** vary `stage_b.iou_thresh` to see trade-offs between strict matching and forgiving coverage.
 - **Crop size impact:** test `crop_size = null` vs `(256,256)`; larger crops may retain more context but increase storage.
 - **Semantic availability:** run with and without head manifests to quantify effect of positive supervision density.
- Reporting guidance:
 - Plot `recall_at_K` vs `K`; annotate runtime and storage to argue for a chosen operating point.
 - Include qualitative crops for failure analysis (false positives, off-frame boxes).

14. Reproducibility, Logging, and Run Naming

- Run naming convention:
 - Include mode, `K`, and date in the Stage A prefix, e.g., `trackA_stageA_oracle_K5_2025-11-26`.
 - Stage B run prefix can mirror Stage A to pair results easily, e.g., `trackA_stageB_oracle_K5_2025-11-26`.

- Provenance tracking:
 - `summary.json` in both stages captures configuration snapshots; Stage B also embeds detector config when available.
 - Recall metrics can be mirrored back to Stage A summary to keep a single source of truth per detector run.
- Logging helper:
 - Both stages attempt to use `core.RunLogger` to write configs, metrics, and artifacts. If unavailable, runs still succeed and print a warning.
- Version control etiquette:
 - Avoid committing large run folders; keep them in `local_extraction/runs/Track_A/` and reference paths in the thesis text.
 - Document overrides used for each figure/table to ensure exact reproducibility.

15. Performance, Resource Management, and Determinism

- Compute considerations:
 - YOLO inference benefits from GPU acceleration; set the Ultralytics device via environment if needed.
 - Cropping in Stage B is CPU-bound; parallelization is not implemented in this script, so adjust `K` and `max_images` to fit time budgets.
- I/O considerations:
 - Reading frames and writing crops dominate disk throughput. SSDs reduce runtime significantly.
 - Per-image CSVs in Stage A are optional; disable them for faster runs and smaller footprints.
- Memory considerations:
 - Images are loaded one at a time; memory usage remains modest even for large datasets.
 - Caching is minimal; repeated runs reread from disk, which keeps results deterministic but not optimized for speed.
- Determinism:
 - File iteration is sorted to stabilize ordering.
 - No random shuffling occurs in Stage A or B.
 - To propagate determinism downstream, fix seeds in Track B training and record configuration hashes.

16. Troubleshooting Playbook

- **Imports fail immediately:**
 - Ensure the venv is activated and `python` points to it.
 - Run quick import checks as shown in the setup workflow.
- **Ultralytics not installed:**
 - Install inside venv: `pip install ultralytics`.
 - As a fallback, switch to oracle mode to continue experiments without detector inference.
- **Frames not found:**
 - Verify `version` and `paths.extracted_frames` in `trackA.yaml`.
 - Inspect the directory: `ls local_extraction/v2/extracted_frames | Measure-Object`.
- **Stage B uses the wrong Stage A run:**
 - Set `stage_b.candidates_source` explicitly to the desired `candidates.jsonl`.
- **Recall is unexpectedly low:**

- Confirm `label_space` consistency.
- Increase `K` or lower `conf_thresh` in YOLO mode.
- Inspect `best_iou` histogram in `summary.json` for distribution skew.
- **Crops are empty or very small:**
 - Candidate boxes may be degenerate; inspect clamped `x1..y2` values in `manifest.jsonl` or visualize crops.
 - Consider raising `conf_thresh` to filter noise.
- **Head manifests missing or mismatched:**
 - Check manifest file paths and ensure `(uid, frame)` keys align with extracted frames.
 - Verify whether you need clip-level or video-level manifests and set `use_clip_manifest` accordingly.
- **Disk usage grows quickly:**
 - Reduce `K`, disable per-image CSVs, and clean older runs in `runs/Track_A/`.

17. Appendix A — Parameter Reference (Stage A)

- `version` : data version; default `v2`.
- `paths.extracted_frames` : root for frames; default `local_extraction/v2/extracted_frames`.
- `paths.yolo_labels` : root for YOLO labels; default `local_extraction/v2/yolo_labels_540`.
- `stage_a.mode` : "yolo" or "oracle".
- `stage_a.k` : max proposals per image.
- `stage_a.last_frame_only` : process only the last frame per `uid` when `true`.
- `stage_a.max_images` : cap on processed images; `null` for all.
- `stage_a.yolo.weights` : path to YOLO checkpoint.
- `stage_a.yolo.imgsz` : inference size.
- `stage_a.yolo.conf_thresh` : detection confidence threshold.
- `stage_a.yolo.nms_iou` : non-max suppression IoU threshold.
- `stage_a.oracle.label_space` : choose between `clips` and `videos`.
- `stage_a.output.save_per_image_csv` : write per-image CSVs when `true`.
- `stage_a.output.run_prefix` : prefix for run directory naming.
- `demo.enabled`, `demo.max_samples` : optional quick-mode limits shared from base config.
- `runtime.print_progress` : enable tqdm or simple progress output.

18. Appendix B — Parameter Reference (Stage B)

- `version` : data version; default `v2`.
- `paths.extracted_frames` : root for frames; shared with Stage A.
- `paths.yolo_labels` : root for labels used in evaluation.
- `stage_b.candidates_source` : explicit path to `candidates.jsonl`; `null` for auto-detect latest.
- `stage_b.crop_size` : [W, H] or `null` (no resize).
- `stage_b.keep_top_n` : optional limit on candidates per image.
- `stage_b.eval_with_labels` : enable recall computation using YOLO labels.
- `stage_b.iou_thresh` : IoU threshold τ for hits and positives.
- `stage_b.manifests.use_clip_manifest` : choose clip-level vs video-level semantics.
- `stage_b.manifests.train` / `stage_b.manifests.val` : paths to semantic manifests.
- `stage_b.output.write_head_train_val` : emit consolidated head files when `true`.
- `stage_b.output.write_semantics_in_manifest` : append semantic fields to manifest rows.

- `stage_b.output.write_back_to_stagea_summary` : patch Stage A summary with recall metrics.
- `stage_b.output.run_prefix` : prefix for Stage B run directory.
- `runtime.max_images` : cap on images processed in Stage B.
- `demo.enabled` , `demo.max_samples` : quick mode limits.
- `runtime.print_progress` : enable tqdm or periodic prints.

19. Appendix C — File Schemas and Examples

- **Stage A candidates (`candidates.jsonl`):**

- Schema: `{ "uid": str, "frame": int, "boxes": [ { "x1": float, "y1": float, "x2": float, "y2": float, "conf": float, "cls": int } ] }`
- Example:

```
{ "uid": "03abc", "frame": 120, "boxes":
  [ { "x1": 123.4, "y1": 56.7, "x2": 200.1, "y2": 180.0, "conf": 0.91, "cls": 0 } ] }
```

- **Stage B manifest (`manifest.jsonl`):**

- Base fields: `uid`, `frame`, `roi_idx`, `crop_path`, `x1`, `y1`, `x2`, `y2`, `conf`, `cls`, `hit`
- Optional semantic fields: `verb_id`, `noun_id`, `ttc`, `is_positive`, `iou`
- Example:

```
{ "uid": "03abc", "frame": 120, "roi_idx": 0, "crop_path": "crops/03abc/0000
  120_00.jpg", "x1": 120, "y1": 50, "x2": 210, "y2": 190, "conf": 0.91, "cls": 0,
  "hit": true, "verb_id": 7, "noun_id": 42, "ttc": 0.83, "is_positive": true, "io
  u": 0.64 }
```

- **Head files (`head_train.jsonl` / `head_val.jsonl`):**

- Fields: `image_path`, `candidate_box`, `candidate_conf`, `candidate_cls`, `verb_id`, `noun_id`, `ttc`, `is_positive`, `iou`
- Example:

```
{ "image_path": "local_extraction/v2/extracted_frames/03abc/0000120.jp
  g", "candidate_box":
  [ 120, 50, 210, 190 ], "candidate_conf": 0.91, "candidate_cls": 0, "verb_id": 7
  , "noun_id": 42, "ttc": 0.83, "is_positive": true, "iou": 0.64 }
```

- **Summary files (`summary.json`):**

- Stage A: counts, mode, `K`, paths, duration, optional YOLO config.
- Stage B: counts, paths, detector metadata, recall metrics, head manifest bookkeeping.

20. Appendix D — Environment Variable Overrides (Stage A)

- `STAGEA_MODE` : "oracle" or "yolo" .
- `STAGEA_K` : integer for top-K proposals.
- `STAGEA_MAX_IMAGES` : integer cap on images processed.
- `STAGEA_DEMO_MODE` : set to `1` or `true` to enable demo sampling.
- `STAGEA_DEMO_N` : integer number of demo samples.
- Usage pattern:

```
$env:STAGEA_MODE = "oracle"
$env:STAGEA_K = "8"
python local_extraction/trackA/trackA_stageA/trackA_stageA.py
```

- These overrides are best for quick sweeps; for long-running experiments, edit `configs/trackA.yaml` to persist settings.

21. Appendix E — Checklists and Sanity Tests

- **Minimal Stage A run checklist:**
 - ☐ Frames present under `local_extraction/v2/extracted_frames/` .
 - ☐ Detector mode set correctly (YOLO weights available or oracle labels present).
 - ☐ `K` and `last_frame_only` set for the intended recall/runtime balance.
 - ☐ Run completed with non-empty `candidates.jsonl` and `summary.json` .
- **Minimal Stage B run checklist:**
 - ☐ `candidates.jsonl` path resolved (auto or explicit).
 - ☐ Labels present if recall evaluation is desired.
 - ☐ `manifest.jsonl` and `manifest.csv` written.
 - ☐ `head_train.jsonl` and `head_val.jsonl` written when semantics exist.
 - ☐ `summary.json` contains recall metrics when evaluation is enabled.
- **Sanity script:**

```
python local_extraction/trackA/trackA_stageB/san_stageB.py
```

- Prints counts of positives/negatives and recall metrics for the latest Stage B run.
- **Manual spot-check:**
 - Randomly open a crop from `crops/<uid>/` and compare with the source frame to confirm alignment.

22. Appendix F — Integration Notes for Track B and Track C

- Track B expects `head_train.jsonl` and `head_val.jsonl` with `is_positive` labels and TTC values when available.
- If semantics are absent, Track B falls back to binary detection and TTC regression using available fields.
- Track C pruning reuses Stage B outputs and the Track B checkpoint; no additional Track A steps are required once manifests are produced.

- Auto-discovery:
 - Track B uses a helper (e.g., `latest_stageB_run`) to pick the newest Stage B run unless a path is specified.
 - Keep run prefixes descriptive to avoid ambiguity when multiple runs share timestamps close together.
 - When reporting in the thesis:
 - Describe the chosen `K` , IoU threshold, and whether semantics were included.
 - Link Stage A recall (oracle vs YOLO) to Track B performance to argue for the selected operating point.
-

This chapter is intentionally verbose (beyond 400 lines) to serve as a single, self-contained reference for Track A. It captures both the conceptual framing and the precise commands and paths needed to regenerate every artifact used by downstream tracks. Adjust sections locally if a shorter version is desired for publication.